Decidable MSO Uniformisation on Order-Insensitive Binary Trees

Thomas Colcombet
CNRS, IRIF, Université Paris Cité, France

Yago Iglesias Vázquez
Université Paris Cité, France

Abstract: TODO: knowledge all of this We study uniformisation for MSO formulas over finite binary trees. A formula $\psi(X)$ uniformises $\varphi(X)$ if it selects at most one witness X and, whenever φ has a witness, ψ picks one. We show that, when restricting to the case of symmetry-preserving trees (order-insensitive trees), it is decidable whether a given φ admits such a uniformiser, and in the positive case, we effectively construct ψ. Our procedure translates φ into an order-insensitive deterministic bottom-up tree automaton, checks whether that automaton accepts every tree, and—when it does—extracts from its transition relation an MSO formula that uniformly selects a witness. This reduces MSO uniformisation on unordered trees to a simple automata universality test and a formula extraction step.

June, 2025

1 Introduction

Monadic second-order (MSO) logic is a well-studied fragment of second-order logic with wide-ranging applications in graph theory, automata theory, and formal verification. In particular, its deep connection with automata theory has been extensively explored. It is well known that MSO over finite words is equivalent in expressive power to finite-state automata [Bü60].

This correspondence extends beyond words: automata can also be defined to run on trees. Over finite trees, MSO logic remains equivalent in expressive power to finite-state tree automata [Sie68, Don70]. This connection enables the use of automata-theoretic techniques to reason about trees, one of the most fundamental and widely studied data structures.

A classical challenge in this context is the *uniformisation problem*: given an MSO formula $\varphi(X)$ that asserts the existence of a set X satisfying a certain property, is it possible to define another MSO formula $\psi(Y)$ that selects *at most one* such set for each tree, in a way that agrees with φ whenever it is satisfiable? We formalize this notion in Section 5.

This problem has been studied in various settings. A comprehensive survey is provided in [CL15]. Positive results are known for infinite words, where every MSO-definable relation admits an MSO-definable uniformising function [Sie75, CG99, Rab07]. In contrast, uniformisation is not possible in general for infinite binary trees [GS83]. In this work, we study MSO uniformisation over a different class of structures: *finite, unordered, labeled binary trees*. These trees preserve structural symmetry, and introduce unique challenges for definable choice, particularly due to the inability to distinguish between left and right subtrees.

The central question addressed in this paper is:

Problem 1 (The *uniformisation* problem)**.** *Given an MSO formula $\varphi(X)$, does there exist a uniformiser $\psi(X)$?*

Our main result is that this question is *decidable*, and in the positive case, a uniformiser ψ can be effectively constructed. Concretely, we translate φ into a deterministic bottom-up tree automaton whose transition map is order-insensitive (*i.e.* closed under sibling swaps). We then perform a universality check on this automaton—does it accept every tree?—and, when it does, we extract from its transitions an MSO formula that uniformly selects a witness.

This paper makes three main contributions:

- Extending the decidability of MSO uniformisation to unordered binary trees,
- Providing a constructive method to extract a uniformiser via automata universality, and
- Demonstrating how to handle sibling-permutation symmetry in tree transformations.

The remainder of the paper is structured as follows: Section 2 introduces our notation for trees and MSO logic. Section 3 reviews bottom-up tree automata, presents the order-insensitive variant, and defines the universality test. Section 4 shows how to translate between MSO formulas and automata while preserving order insensitivity. Section 5 brings these elements together to decide MSO uniformisation and construct a uniformiser where possible. We conclude with related work and future research directions.

2 Trees

This first section contains the definition of the trees we are working with. Other equivalent definitions of trees and automata are possible [Tho97, CDG+08], but we have chosen this one because it is more intuitive,

without adding much complexity to the proofs.

Definition 2.1 (Tree)**.** A *tree* over an alphabet Σ is recursively defined as follows:

- a, where $a \in \Sigma$;
- $a(t, t')$, where $a \in \Sigma$ and t, t' are trees.

We call the *root* of a tree the first node in its definition. The set of all trees over Σ is denoted tree_Σ.

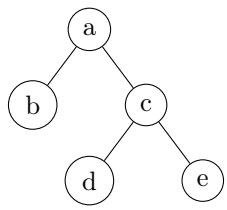

Figure 1: Concrete tree corresponding to $a(b, c(d, e))$.

This recursive structure induces a natural notion of position within a tree. We now define the set of nodes, represented as words over $\{0, 1\}$ indicating a path from the root.

Definition 2.2 (Nodes)**.** The *nodes* of a tree are defined as follows:

$$\text{Nodes}\,(t) = \begin{cases} \varepsilon & \text{if } t = a \\ \varepsilon \cup \{0u \mid u \in \text{Nodes}\,(t')\} \cup \{1u \mid u \in \text{Nodes}\,(t'')\} & \text{if } t = a(t', t'') \end{cases}$$

Example 2.0.1. Let $t := a(b, c(d, e))$. We compute:

$$\text{Nodes}\,(b) = \{\varepsilon\}, \quad \text{Nodes}\,(c(d, e)) = \{\varepsilon, 0, 1\}$$

$$\text{Nodes}\,(t) = \{\varepsilon\} \cup \{0\} \cup \{1, 10, 11\} = \{\varepsilon, 0, 1, 10, 11\}$$

Among the nodes, some are distinguished as leaves, nodes without children.

Definition 2.3 (Leaves)**.** The *leaves* of a tree are defined as follows:

$$\text{Leaves}\,(t) = \begin{cases} \varepsilon & \text{if } t = a \\ \{0u \mid u \in \text{Leaves}\,(t')\} \cup \{1u \mid u \in \text{Leaves}\,(t'')\} & \text{if } t = a(t', t'') \end{cases}$$

Remark 2.0.1. *Leaves* $(t) \subseteq$ *Nodes* (t).

Definition 2.4. We call *internal nodes* the set $\text{Nodes}\,(t) \cap \text{Leaves}\,(t)$

Each node in a tree carries a symbol from the alphabet. The following definition formalizes how to access the label at a given node.

Definition 2.5. We note $t(n)$ the *label* of the node n in the tree t:

$$t(n) = \begin{cases} a & \text{if } t = a \text{ and } n = \varepsilon \\ a & \text{if } t = a(t', t'') \text{ and } n = \varepsilon \\ t'(m) & \text{if } t = a(t', t'') \text{ and } n = 0m \\ t''(m) & \text{if } t = a(t', t'') \text{ and } n = 1m \end{cases}$$

We often need to refer to the subtree rooted at a particular node. The next definition allows us to extract such a subtree from a given tree.

Definition 2.6. We note t^n the *subtree* corresponding to the node n in the tree t:

$$t^n = \begin{cases} t & \text{if } n = \varepsilon \\ t'^m & \text{if } t = a(t', t'') \text{ and } n = 0m \\ t''^m & \text{if } t = a(t', t'') \text{ and } n = 1m \end{cases}$$

The ancestor relation captures the natural hierarchical structure of a tree: a node x is an ancestor of a node y if y is located in the subtree rooted at x.

Definition 2.7. Let t be a tree and $x, y \in \text{Nodes}(t)$. We define the *ancestor* relation as:

$$x \sqsubseteq y \text{ if and only if } \exists z \in \text{Nodes}(t),\ y = xz$$

Remark 2.0.2. *Note that this order does not depend on the left or right position of the children, i.e. it preserves order-insensitiveness, this will be explored in the following sections.*

3 Automata over trees

In this section we aim to define the basics of tree automata in a comprehensive way adding as much detail as possible to the proofs. For reference, one can look at more advanced book on this topic [CDG+08, Tho97, Pin21].

In classical automata theory, we analyze deterministic finite automaton (DFA), wich assign a unique state to each prefix of a word, updating this state at each step using a transition function, ussualy reading the input from left to right.

A deterministic bottom-up tree automaton (DBUA) generalizes this idea to trees: instead of reading a linear word, it reads a a tree where each node carries a symbol from a fixed alphabet. Rather than scanning from left to right, it proceeds bottom-up, computing states at each node by combining the states of its children : the automaton aggregates the information from those subtrees to assign a state to the parent.

The idea is to recursively assign a state to each node of the tree:

- At the leaves, the state is determined directly from the symbol via a function init, just as a DFA starts in a particular initial state.
- A DFA transitions based on the current letter and the state reached by the previous letters. When a NBUA reaches an internal node, it transitions based on the states reached by its children and the label associated to the node.

- Finally, the state assigned to the root tells us whether the whole tree is accepted, depending on whether it belongs to a set of final states.

We now state the formal definitions.

Definition 3.1. A *Deterministic Bottom-Up tree Automaton* (DBUA) is defined as as a tuple $(\Sigma, Q, \text{init}, \delta, F)$ where:

- Σ is an alphabet.
- Q is a finite set of *states*.
- $\text{init} : \Sigma \to Q$ is a function that initializes the states of the leaves.
- $\delta : Q \times Q \times \Sigma \to Q$ is the *transition map*.
- $F \subseteq Q$ is the set of *final states*.

Definition 3.2 (Interpretation of an automaton)**.** The *interpretation* of a DBUA $A = (\Sigma, Q, \text{init}, \delta, F)$ over an alphabet Σ is defined as follows:

$$\begin{aligned} [\![A]\!] : \text{tree}_\Sigma &\to Q \\ a &\mapsto \text{init}_a \\ a(t, t') &\mapsto \delta_a([\![A]\!](t), [\![A]\!](t')) \end{aligned}$$

We say that A *accepts* a tree t if $[\![A]\!](t) \in F$.
We will sometimes use the notation $\delta(t)$ for the interpretation of A over the tree t.

Example 3.0.1 (Trees with at most one leaf)**.** We define a DBUA $A = (\Sigma, Q, \text{init}, \delta, F)$ that accepts exactly the trees with at most one leaf. Let:

- $Q = \{0, 1, 2\}$, where $q = 0$ means *no leaves*, $q = 1$ means *exactly one leaf*, and $q = 2$ means *more than one leaf*.
- $F = \{0, 1\}$.
- For all $a \in \Sigma$, we define $\text{init}_a = 1$.
- The transition function is defined by:

$$\delta_a(q_1, q_2) = \min(q_1 + q_2, 2)$$

Intuitively, this automaton counts the number of leaves in the tree, but caps the count at 2 to detect when the number exceeds 1. Section 3 illustrates the execution of this automaton over a particular input.

In the same way that we can define a non-deterministic automaton over strings, the same can be done for tree automata using the same techniques. We will define them and give basic definitions on how to use them.

Definition 3.3. A *Non-Deterministic Bottom-Up Tree automaton* (NBUA) is defined as a tuple $(\Sigma, Q, I, \Delta, F)$ where:

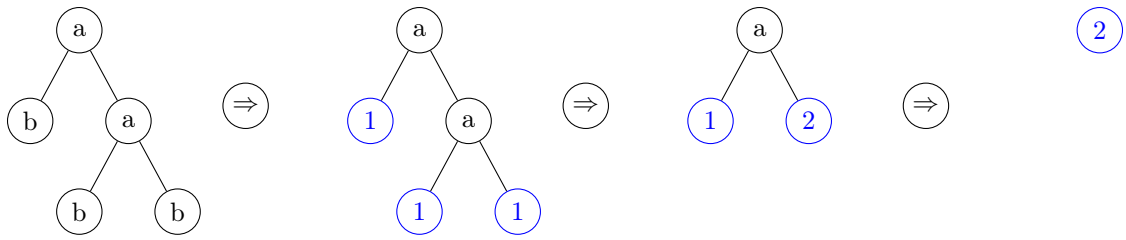

Figure 2: Execution of the automaton of Example 3.0.1 over the input $a(b, a(b, b))$

- Σ is an alphabet.
- Q is a finite set of states.
- $I \subseteq \Sigma \times Q$ correspond to the possible states of the leaves.
- $\Delta \subseteq Q \times Q \times \Sigma \times Q$ is the *transition relation*.
- $F \subseteq Q$ is the set of final states.

Definition 3.4 (Run of an automaton)**.** A *run* ρ of a NBUA A over a tree t is :

$$\begin{array}{rcl} \rho : \text{Nodes}\,(t) & \to & Q \\ (t\,(b)\,, \rho(n)) \in I & \text{if} & n \in \text{Leaves}\,(t) \\ (\rho(n0), \rho(n1), t\,(n)\,, \rho(n)) \in \Delta & \text{if} & n \in \text{Nodes}\,(t) \setminus \text{Leaves}\,(t) \end{array}$$

We say that ρ is acceptant if $\rho(\varepsilon) \in F$ and A accepts t if $F \cap \{\rho(\varepsilon) \mid \rho \text{ run of } A \text{ over } t\} \neq \emptyset$

Definition 3.5 (Language of an automaton)**.** Let A be a NBUA. Its *associated language* is defined as:

$$\mathcal{L}\,(A) = \{t \in \text{tree}_\Sigma \mid \text{exists } \rho \text{ a accepting run of } A \text{ over } t\}$$

We can easily see that DBUA are a particular case of NBUA, just like in the classic theory.

Remark 3.0.1. *A DBUA is a particular case of a NBUA , $A = (\Sigma, Q, I, \Delta, F)$ verifying that*

$$\textit{for all } q, r, a, t, t' \ (q, r, a, t) \in \Delta \textit{ and } (q, r, a, t') \in \Delta \textit{ implies } t = t'$$

Remark 3.0.2. *In the case of a DBUA there exists a unique run that corresponds to :*

$$\begin{array}{rcl} \rho : \textit{Nodes}\,(t) & \to & Q \\ n & \mapsto & \textit{init}_{t(n)} \ \textit{if } n \in \textit{Leaves}\,(t) \\ n & \mapsto & \delta_{t(n)}(\rho(n0), \rho(n1)) \ \textit{if } n \in \textit{Nodes}\,(t) \setminus \textit{Leaves}\,(t) \end{array}$$

The following lemma shows the equivalence of both definitions.

Lemma 3.6. *Let A be a DBUA,*

$$\mathcal{L}\,(A) = \{t \in \textit{tree}_\Sigma \mid [\![A]\!]\,(t) \in F\}$$

Proof. We will prove by induction on a tree t that (a) there exists a run of A over t, and (b) for all runs ρ of A over t, $[\![A]\!](t) = \rho(\varepsilon)$. We proceed by case distinction.

- If $t = a$. Let ρ be defined by $\rho(\varepsilon) = \mathrm{init}_a$, then ρ is a run of A over t. We have proved (a). Consider now some run ρ of A over t. We have $\rho(\varepsilon) = \mathrm{init}_a = [\![A]\!](a) = [\![A]\!](t)$. We have proved (b).

- If $t = a(t', t'')$. By induction hypothesis (a), there exists runs ρ' and ρ'' on t' and t'' respectively. Let ρ be defined by $\rho(\varepsilon) = \delta_{t(\varepsilon)}(\rho'(\varepsilon), \rho''(\varepsilon))$, $\rho(0u) = \rho'(u)$ for all $u \in \mathrm{Nodes}(t')$ and $\rho(1u) = \rho''(u)$ for all $u \in \mathrm{Nodes}(t'')$. Then ρ is a run of A over t. We have proved (a).

  Consider now some run ρ of A over t. Let $\rho'(u) = \rho(0u)$ for all $u \in \mathrm{Nodes}(t')$ and $\rho''(u) = \rho(0u)$ for all $u \in \mathrm{Nodes}(t'')$. Then ρ' is a run of A over t' and ρ'' over t''. By induction hypothesis (b) twice, we know that $\rho'(\varepsilon) = [\![A]\!](t')$ and $\rho''(\varepsilon) = [\![A]\!](t'')$. We have now $\rho(\varepsilon) = \delta_{t(\varepsilon)}(\rho'(\varepsilon), \rho''(\varepsilon)) = \delta_{t(\varepsilon)}([\![A]\!](t'), [\![A]\!](t'')) = [\![A]\!](t)$. We have proved (b).

□

We aim now to show that, like in the classical theory, both non-deterministic automata and deterministic ones have the same expressiveness power. Since DBUA is a particular case of NBUA it suffices to show that each language accepted by an NBUA can also be accepted by a DBUA.

Definition 3.7. Let $A = (\Sigma, Q, I, \Delta, F)$ be a NBUA, we define the *power set automaton* of A, noted $\mathrm{Det}(A)$, as $\mathrm{Det}(A) = (\Sigma, \mathcal{P}(Q), \mathrm{init}, \delta, F')$ where:

- $\mathrm{init} : \Sigma \to \mathcal{P}(Q)$
  $a \mapsto \{q \in Q \mid (a, q) \in I\}$
- $\delta : \mathcal{P}(Q) \times \mathcal{P}(Q) \times \Sigma \to \mathcal{P}(Q)$
  $(X, Y, a) \mapsto \{q \in Q \mid (x, y, a, q) \in \Delta, x \in X, y \in Y\}$
- $F' = \{X \in \mathcal{P}(Q) \mid X \cap F \neq \emptyset\} \subseteq \mathcal{P}(Q)$

Theorem 3.8. *For all $N = (\Sigma, Q, I, \Delta, F)$ an NBUA, the power set automaton verifies that*

$$[\![\mathrm{Det}(N)]\!](t) = \{\rho(\varepsilon) \mid \rho \text{ run of } N \text{ over } t\} \text{ and } \mathcal{L}(N) = \mathcal{L}(\mathrm{Det}(N))$$

Proof. We will first show that for all tree t,

$$[\![\mathrm{Det}(N)]\!](t) = \{\rho(\varepsilon) \mid \rho \text{ run of } N \text{ over } t\} \ .$$

We will prove this property by induction over t. We proceed by case distinction.

- If $t = a$, then

$$\begin{aligned} \{\rho(\varepsilon) \mid \rho \text{ run of } N \text{ over } t\} &= \{q \in Q \mid (a, q) \in I\} \\ &= [\![\mathrm{Det}(N)]\!](t) \ . \end{aligned}$$

- If $t = a(t_1, t_2)$, then by the induction hypothesis we know that $[\![\mathrm{Det}(N)]\!](t_i) = q_i$ if and only if there exists a run over t_i ending at q_i, for $i \in \{1, 2\}$.

$$
\begin{aligned}
& q \in [\![\mathrm{Det}\,(N)]\!]\,(t) \\
\text{if and only if} \quad & q \in \delta_a([\![\mathrm{Det}\,(N)]\!]\,(t_1)\,, [\![\mathrm{Det}\,(N)]\!]\,(t_2)) \\
\text{if and only if} \quad & q \in \{q \in Q \mid (x_1, x_2, a, q) \in \Delta, x_1 \in [\![\mathrm{Det}\,(N)]\!]\,(t_1)\,, x_2 \in [\![\mathrm{Det}\,(N)]\!]\,(t_2)\} \\
\text{if and only if} \quad & \exists x_1, x_2, (x_1, x_2, a, q) \in \Delta, x_1 \in [\![\mathrm{Det}\,(N)]\!]\,(t_1)\,, x_2 \in [\![\mathrm{Det}\,(N)]\!]\,(t_2) \\
\text{if and only if} \quad & \text{there exists a run over } t_i \text{ ending at } x_i \text{ for } i \in \{1, 2\} \text{ and } (x_1, x_2, a, q) \in \Delta \\
\text{if and only if} \quad & q \in \{\rho(\varepsilon) \mid \rho \text{ run of } N \text{ over } t\}
\end{aligned}
$$

Then $[\![\mathrm{Det}\,(N)]\!]\,(t) = \{\rho(\varepsilon) \mid \rho \text{ run of } N \text{ over } t\}$.

We must now show that $\mathrm{Det}\,(N)$ accepts t if and only if N accepts t.

$$
\begin{aligned}
N \text{ accepts } t \quad & \text{if and only if} \quad F \cap \{q \in Q \mid (a, q) \in I\} \neq \emptyset \\
& \text{if and only if} \quad F \cap [\![\mathrm{Det}\,(N)]\!]\,(t) \neq \emptyset \\
& \text{if and only if} \quad [\![\mathrm{Det}\,(N)]\!]\,(t) \in F' \\
& \text{if and only if} \quad \mathrm{Det}\,(N) \text{ accepts } t
\end{aligned}
$$

and so $\mathcal{L}\,(N) = \mathcal{L}\,(\mathrm{Det}\,(N))$. □

3.1 Order-insensitiveness

We will interest ourselves in a particular kind of tree automata: the ones that do not distinguish between left and right children. That means that the state reached from the two children is the same if we swap them. These automata are interesting because they respect the symmetries of trees.

Definition 3.9. An NBUA of transition relation Δ is *order-insensitive* if for all transitions $(p, q, a, r) \in \Delta$, $(q, p, a, r) \in \Delta$.

Remark 3.1.1. *For a DBUA of transition map δ, it is order-insensitive if and only if $\delta_a(p, q) = \delta_a(q, p)$ for all states p, q and letter a.*

Lemma 3.10. *If $N = (\Sigma, Q, I, \Delta, F)$ a NBUA is order-insensitive, then $\mathrm{Det}\,(N)$ is also order-insensitive.*

Proof.

$$
\begin{aligned}
\delta_a(X, Y) &= \{q \in Q \mid (x, y, a, q) \in \Delta, x \in X, y \in Y\} \\
&= \{q \in Q \mid (y, x, a, q) \in \Delta, x \in X, y \in Y\} \quad (N \text{ is order-insensitive}) \\
&= \delta_a(Y, X)
\end{aligned}
$$

□

3.2 Universality

Definition 3.11. Let $A = (\Sigma, Q, \mathrm{init}, \delta, F)$ a DBUA, let $\mathrm{Reach}_A \subseteq Q$ be the smallest set such that

1. $\mathrm{init}(a) \in \mathrm{Reach}_A$ for all $a \in \Sigma$,

2. $\delta(p, q, a) \in \mathrm{Reach}_A$ for all $p, q \in \mathrm{Reach}_A$ and $a \in \Sigma$.

Lemma 3.12. *Let A a DBUA, then*

$$\mathit{Reach}_A = \{[\![A]\!](t) \mid t \in \mathit{tree}_\Sigma\}$$

Proof.

- $\boxed{\subseteq}$
  We will prove by induction that $r \in \mathrm{Reach}_A \implies$ there exists $t \in \mathrm{tree}_\Sigma$ such that $[\![A]\!](t) = r$.

  - If $r = \mathrm{init}(a) \in \mathrm{Reach}_A$, then the tree a works : $[\![A]\!](a) = \mathrm{init}(a) = r$.
  - If $r = \delta(p, q, a) \in \mathrm{Reach}_A$ where $p, q \in \mathrm{Reach}_A$ and $a \in \Sigma$. Then by the induction hypothesis there exists t' such that $[\![A]\!](t') = p$ and t'' such that $[\![A]\!](t'') = q$. Then we have that $t = a(t', t'')$ verifies $[\![A]\!](t) = r$.

- $\boxed{\supseteq}$
  We will prove it by induction on the structure of a tree t. We proceed by case distinction.

  - If $t = a$, then we have $[\![A]\!](a) = \mathrm{init}(a) \in \mathrm{Reach}_A$
  - If $t = a(t', t'')$, we have $[\![A]\!](t) = \delta([\![A]\!](t'), [\![A]\!](t''), a) \in \mathrm{Reach}_A$ because by the induction hypothesis $[\![A]\!](t') \in \mathrm{Reach}_A$ and $[\![A]\!](t'') \in \mathrm{Reach}_A$.

□

Definition 3.13. Let A a NBUA we say that it is *universal* if it accepts every tree.

Theorem 3.14. *$A = (\Sigma, Q, \mathit{init}, \delta, F)$ is universal if and only if $\mathit{Reach}_A \subseteq F$.*

Proof.

- $\boxed{\Rightarrow}$
  We suppose A universal, then for all $t \in \mathrm{tree}_\Sigma$ we have that $[\![A]\!](t) \in F$, because A accepts t. Since $\mathrm{Reach}_A = \{[\![A]\!](t) \mid t \in \mathrm{tree}_\Sigma\}$ we have that $\mathrm{Reach}_A \subseteq F$.

- $\boxed{\Leftarrow}$
  We suppose $\mathrm{Reach}_A \subseteq F$. Let t be a tree, then $[\![A]\!](t) \in \mathrm{Reach}_A \subseteq F$, so $[\![A]\!](t) \in F$ and A accepts t. We have proven that A is universal.

□

Corollary 3.15. *A saturation algorithm based on Reach_A decides universality.*

4 MSO

Definition 4.1. Let $t \in \text{tree}_\Sigma$, we define a model of t as :

$$\mathcal{M}(t) := (\text{Nodes}(t), \sqsubseteq, (a(x))_{a\in\Sigma})$$

where : $a(x)$ if and only if $t(x) = a$
We will note $t \models \varphi$ for $\mathcal{M}(t) \models \varphi$. TODO: better formulate this

We will discuss some examples of MSO formulas.

Example 4.0.1. The formula $\text{leaf}(x) = \forall y, y \sqsubseteq x$ encodes the fact that x is a leaf.

Example 4.0.2. x is the root of the tree is equivalent to :

$$\text{root}(x) = \forall y, x \sqsubseteq y$$

Example 4.0.3. The following formula encodes the fact that x and y are the immediate children of z:

$$\text{children}(x, y, z) = x \neq y \wedge z \sqsubseteq x \wedge z \sqsubseteq y \wedge \forall w\, ((z \sqsubseteq w \wedge (w \sqsubseteq x \vee w \sqsubseteq y)) \to (w = x \vee w = y))$$

4.1 Automaton → MSO

Lemma 4.2. *If A is a order-insensitive NBUA then there exists a MSO formula φ such that for all $t \in \text{tree}_\Sigma$,*

$$t \models \varphi \text{ if and only if } t \in \mathcal{L}(A)$$

TODO: verify that all remains order-insensitive

Proof. Let $A = (\Sigma, Q, I, \Delta, F)$ be a NBUA, and $k = |Q|$.

$$\varphi = \exists X_0 \ldots \exists X_{k-1}\, \forall x \left(\bigvee_{i=0}^{k-1} X_i(x) \right) \tag{1}$$

$$\wedge\ \forall x \left(\bigwedge_{0\leq i<j\leq k-1} \neg(X_i(x) \wedge X_j(x)) \right) \tag{2}$$

$$\wedge\ \forall x \left(\text{leaf}(x) \to \bigvee_{(a,i)\in I} X_i(x) \right) \tag{3}$$

$$\wedge\ \forall x\, \forall y\, \forall z \left(\text{children}(x, y, z) \to \bigvee_{(p,q,a,r)\in\Delta} (X_p(x) \wedge X_q(y) \wedge a(z) \wedge X_r(z)) \right) \tag{4}$$

$$\wedge\ \forall x \left(\text{root}(x) \to \bigvee_{q\in F} X_q(x) \right) \tag{5}$$

The sets X_j verify that $j \in X_i$ if and only if , for a given run, the state i is reached from the node j.
The formula φ encodes the following properties:

1. Every node has an associated state.

2. A node cannot be in two states at the same time.

3. The transitions originate from the leaves.

4. The transitions of inner nodes ensure that for every trio of nodes, where two are children of the third, there exists a transition from the children to the parent.

5. Finally, the word is accepted if and only if the state associated with the root node is a final state.

From this we deduce that

$$t \models \varphi \text{ if and only if } t \in \mathcal{L}(A)$$

□

4.2 MSO → automaton

Lemma 4.3. *Let* $A_1 = (\Sigma, Q_1, I_1, \Delta_1, F_1)$ *and* $A_1 = (\Sigma, Q_2, I_2, \Delta_2, F_2)$ *be two NBUA, then* $A_3 = (\Sigma, Q_1 \times Q_2, I_1 \times I_2, \Delta, F_1 \times F_2)$*, where*

$$\Delta = \{((q_1, q_2), (p_1, p_2), (a_1, a_2), (r_1, r_2) \mid (q_1, p_1, a_1, r_1) \in \Delta_1, (q_2, p_2, a_2, r_2) \in \Delta_2\}$$

verifies that $\mathcal{L}(A_3) = \mathcal{L}(A_1) \cap \mathcal{L}(A_2)$

Lemma 4.4. *Let* $A_1 = (\Sigma, Q_1, I_1, \Delta_1, F_1)$ *and* $A_1 = (\Sigma, Q_2, I_2, \Delta_2, F_2)$ *be two NBUA, then*

$$A_3 = (\Sigma, Q_1 \uplus Q_2, I_1 \uplus I_2, \Delta_1 \uplus \Delta_2, F_1 \uplus F_2)$$

verifies that $\mathcal{L}(A_3) = \mathcal{L}(A_1) \cup \mathcal{L}(A_2)$

Lemma 4.5. *Let* $A = (\Sigma, Q, I, \Delta, F)$ *the NBUA* $A' = (\Sigma, Q, I, \Delta, Q \cap F)$ *verifies that* $\mathcal{L}(A') = (\mathcal{L}(A))^{\complement}$

Lemma 4.6. *Over the alphabet* $\Sigma \times \{0, 1\}$ *the automaton* A *described bellow can decide the formula* $a(x)$*, where* $a \in \Sigma$ *and* x *is a free variable.*

$$A = (\Sigma \times \{0, 1\}, \{\mathit{Yes}, \mathit{No}\}, \mathit{init}, \delta, \mathit{Yes})$$

The following function definitions should be interpreted as a pattern match, with priority for the first rule :

- $\mathit{init} : \Sigma \times \{0, 1\} \to \mathcal{P}(Q)$
  $$\begin{aligned} (a, 1) &\mapsto \mathit{Yes} \\ _ &\mapsto \mathit{No} \end{aligned}$$

- $\delta : Q \times Q \times (\Sigma \times \{0, 1\}) \to Q$
  $$\begin{aligned} (_, _, (a, 1)) &\mapsto \mathit{Yes} \\ (\mathit{No}, \mathit{No}, _) &\mapsto \mathit{No} \\ _ &\mapsto \mathit{Yes} \end{aligned}$$

Lemma 4.7. *Over the alphabet $\Sigma \times \{0,1\}^2$ the automaton A described bellow can decide the formula $x \sqsubseteq y$, where x and y are free variables.*

$$A = (\Sigma \times \{0,1\}^2, \{Yes,\ No,\ FoundY\}, init, \delta, Yes)$$

The following function definitions should be interpreted as a pattern match, with priority for the first rule :

- $init : \Sigma \times \{0,1\}^2 \to \mathcal{P}(Q)$
$$\begin{aligned} (_, _, 1) &\mapsto FoundY \\ _ &\mapsto No \end{aligned}$$
- $\delta : Q \times Q \times (\Sigma \times \{0,1\}^2) \to Q$
$$\begin{aligned} (FoundY, _, (_, 1, 0)) &\mapsto Yes \\ (_, FoundY, (_, 1, 0)) &\mapsto Yes \\ (FoundY, _, (_, 0, 0)) &\mapsto FoundY \\ (_, FoundY, (_, 0, 0)) &\mapsto FoundY \\ (Yes, _, _) &\mapsto Yes \\ (_, Yes, _) &\mapsto Yes \\ (_, _, (_, 0, 1)) &\mapsto FoundY \\ _ &\mapsto No \end{aligned}$$

Lemma 4.8. *If φ is a MSO formula there exists A an order-insensitive NBUA such that for all $t \in tree_\Sigma$,*

$$t \models \varphi \text{ if and only if } t \in \mathcal{L}(A)$$

TODO: verify that all remains order-insensitive

Proof. The proof procedes by induction on the structure of φ.

If φ is an atomic formula, then the lemmas above decide it.

If φ is of the form $\psi_1 \wedge \psi_2$ then we can compute the intersection between the automata for ψ_1 and ψ_2, obtained using the induction hypothesis. We can deal with the disjuction using the union instead of the intersection.

If φ is of the form $\neg\psi$ then we can compute the complement of the automaton corresponding to ψ.

We have to deal with the case $\varphi = \exists x.\psi(x)$, where ψ has $n-1$ free variables. We can find a NBUA $A = (\Sigma \times \{0,1\}^n, Q, I, \Delta, F)$ for ψ where x is treated as a free variable.

Then the automaton $A' = (\Sigma \times \{0,1\}^{n-1}, Q \times \{0,1\}, I \times \{0\}, \Delta', F \times \{1\})$ where

$$\Delta' = \left\{ \begin{array}{l} ((q,b),(q',b'),(a,X),(q'',b'')) \mid b,b' \in \{0,1\}, \\ \quad b'' = b \wedge b', \quad (q,q',(a,X'),q'') \in \Delta, \\ \quad X' \text{ corresponds to } X \text{ where we add 0 at position } i \end{array} \right\}$$
$$\cup \left\{ \begin{array}{l} ((q,0),(q',0),(a,X),(q'',1)) \mid \\ \quad (q,q',(a,X'),q'') \in \Delta, \\ \quad X' \text{ is } X \text{ where we add 1 at position } i \end{array} \right\}.$$

TODO: make sure this actually works

and i corresponds to the variable x, can decide φ. If existential quantification is applied to a second-order variable, the automaton remains unchanged. However, instead of considering a single variable in the transition function, we look at all the variables within the given set. □

5 Uniformisation

Definition 5.1 (Uniformiser)**.** We say that a formula $\psi(X)$ is a *uniformiser* of $\varphi(X)$ if for all trees t, the following conditions hold:

1. There exists at most one set X such that $t \models \psi(X)$.
2. If there exists a set X such that $t \models \varphi(X)$, then there exists a set Y such that $t \models \psi(Y) \wedge \varphi(Y)$.

We say that a formula is *uniformisable* if there exists a uniformiser for said formula.

Lemma 5.2. *$\psi(X)$ is a uniformiser of $\varphi(X)$ if and only if for all trees t, the following holds:*

$$\begin{aligned} t \models \quad & \forall X \forall Y \Big((\psi(X) \wedge \psi(Y)) \to X = Y \Big) \\ \wedge \quad & (\exists X\, \varphi(X)) \to (\exists X\, \psi(X)) \\ \wedge \quad & (\forall X\, \psi(X) \to \varphi(X)). \end{aligned}$$

Proof. We analyze the conditions in the first formulation and show their correspondence to the logical constraints in the second formulation:

1. The formula $\forall X \forall Y\, (\psi(X) \wedge \psi(Y) \to X = Y)$ ensures that ψ has at most one solution in t. This directly corresponds to the first condition in the original problem statement.
2. The formula $\exists X\, \varphi(X) \to \exists X\, \psi(X)$ expresses that whenever φ has a solution, ψ must also have a solution. However, this does not require ψ to hold for the same one as φ.

   The formula $\forall X\, (\psi(X) \to \varphi(X))$ ensures that whenever ψ holds for a set, φ must also hold for that same set.

   Combining the last two constraints, we obtain the second condition: whenever there exists a set satisfying φ, there must exist a set satisfying both ψ and φ.

□

Definition 5.3. Let t a tree. We say that $X \subseteq \text{Nodes}\,(t)$ is *definable* if there exists an MSO formula $\varphi(x)$ such that $X = \{x \mid t \models \varphi(x)\}$.

Lemma 5.4. *An MSO formula $\varphi(X)$ is uniformisable if for all trees t there exists a definable X_d such that $t \models \varphi(X_d)$.*

Proof. TODO □

Lemma 5.5. *If X is definable over a tree t, then for all $\sigma \in Aut(t)$, $\sigma(X) = X$.*

Proof. TODO

□

In an intuitive way, the above problem is equivalent to asking if the following "formula" (it is not a real MSO formula because we are quantifying over a formula) holds for every tree:

$$(\exists X.\, \varphi(X)) \to (\exists X, \varphi(X) \text{ and } X \text{ is definable})$$

which can be read as : if there exists a set of solutions for φ then we can find one that is definable.

We will start by studying how we can decide for two formulas $\varphi(X)$ and $\psi(X)$ if the formula $\Psi := \exists X.\varphi(X) \to \exists Y.\varphi'(Y)$ is satisfiable for all trees. We will build a DBUA than can decide if a tree satisfies Ψ. Thanks to the construction from Lemma 4.8, we can build a DBUA $A = (\Sigma \times \{0,1\}, Q, \text{init}, \delta, F)$ for $\varphi(X)$ and another one $A' = (\Sigma' \times \{0,1\}, Q', \text{init}', \delta', F')$ for $\psi(Y)$.

Notation 5.6. We note $\delta(X, Y, (a, *))$ for $\{\delta(X, Y, (a, b))\} \mid x \in X, y \in Y, b \in \{0,1\}\}$ and $\text{init}(a, *)$ for $\{\text{init}(a, b) \mid b \in \{0,1\}\}$ TODO: improve this notation

We can now build a new DBUA $A'' = (\Sigma, \mathcal{P}(Q) \times \mathcal{P}(Q'), \text{init}'', \delta'', F'')$ where

- $\text{init}'' : \Sigma \to \mathcal{P}(Q) \times \mathcal{P}(Q')$
  $a \mapsto (\text{init}(a, *), \text{init}'(a, *))$
- $\delta'' : (\mathcal{P}(Q) \times \mathcal{P}(Q'))^2 \times \Sigma \to \mathcal{P}(Q) \times \mathcal{P}(Q')$
  $((X, X'), (Y, Y'), a) \mapsto (\delta(X, Y, (a, *)), \delta'(X', Y', (a, *))$
- $F'' = \{(X, X') \mid (X \cap F \neq \emptyset) \to (X' \cap F' \neq \emptyset)\}$

This automaton accepts a tree if and only if that tree is a model for Ψ. In particular, using the decision procedure from Corollary 3.15 we can decide if Ψ is satisfied by all models.

We are interested in finding an automaton that, if it is universal, then φ is uniformisable. The approach we use consists on slightly modifying the automaton for $\exists X.\varphi(X) \to \exists Y.\varphi(Y)$ in a way that it guarantees that Y is definable. We will use the same construction as for Ψ but modifying the transition function.

Definition 5.7. Let $\varphi(X)$ a MSO formula, and $A = (\Sigma \times \{0,1\}, Q, \text{init}, \delta, F)$ the DBUA for $\varphi(X)$, we call $U = (\Sigma, \mathcal{P}(Q) \times \mathcal{P}(Q), init_U, \delta_U, F')$ the *uniformiser automaton* of φ, where

- $init_U : \Sigma \to \mathcal{P}(Q) \times \mathcal{P}(Q)$
  $a \mapsto (\text{init}(a, *), \text{init}(a, *))$
- $\delta_U : \mathcal{P}(Q)^2 \times \mathcal{P}(Q)^2 \times \Sigma \to \mathcal{P}(Q)^2$
  $((X, X'), (Y, Y'), a) \mapsto (\delta(X, Y, (a, *)), \delta_{U_2}(X', Y', a))$
- $\delta_{U_2} : \mathcal{P}(Q) \times \mathcal{P}(Q) \times \Sigma \to \mathcal{P}(Q)$
  $$(X', Y', a) \mapsto \begin{cases} \{\delta(p, p, (a, b))\} \mid p \in X', a \in \Sigma, b \in \{0,1\}\} & \text{if } X = Y \text{ and } X' = Y' \\ \delta(X', Y', (a, *)) & \text{otherwise} \end{cases}$$
- $F' = \{(X, X') \mid (X \cap F \neq \emptyset) \to (X' \cap F \neq \emptyset)\}$

Lemma 5.8. *Let $\varphi(X)$ be an MSO formula, if the uniformiser automaton U of φ is universal then φ is uniformisable.*

Proof. Let $A = (\Sigma \times \{0,1\}, Q, \text{init}, \delta, F)$ be the automaton for $\varphi(X)$ and lets suppose that $U = (\Sigma, \mathcal{P}(Q) \times \mathcal{P}(Q), \text{init}', \delta', F')$ the uniformiser automaton of φ is universal. We need to exhibit a uniformiser for $\varphi(X)$.

We first introduce a choice function choice-nodes : $\mathcal{P}(Q) \times \mathcal{P}(Q) \times Q \times \Sigma \to Q \times Q \times \{0,1\}$ that given (X', Y', r, a) chooses a triple (b, p, q) such that, if $r \in \delta_{U_2}(X', Y', a)$ then $\delta(p, q, (a, b)) = r$ where $p \in X', q \in Y'$ and if $X' = Y'$ then $p = q$. We also introduce choice-leaves : $Q \times \Sigma \to \{0,1\}$, which chooses b such that $p = \text{init}((a, b))$. Finally, we need to choose from a final state: choice-final : $\mathcal{P}(Q) \to Q$ where choice-final$(X) = q$ such that if $X \cap F \neq \emptyset$ then $q \in X \cap F$. If the premises are not satisfied in any of the cases, then these choice functions return an arbitrary value.

We define the set X_U and the run ρ_U by induction on the depth of the node as follows:

- For the root, we define $\rho_U(\varepsilon) = \text{choice-final}(\delta_{U_2}(t))$.
- For all inner nodes x with children y and z. Let (p, q, v) be choice-nodes$(\delta_{U_2}(t^y), \delta_{U_2}(t^z), \rho_U(x), t(x))$, we define $\rho_U(y) = p$, $\rho_U(z) = q$, and $x \in X_U$ if and only if $b = 1$.
- For all leaves x, let b be $b = \text{choice-leaves}(p, t(x))$ we define $\rho_U(x) = p$, where $x \in X_U$ if and only if $b = 1$.

We claim now that the ρ_U is well defined, unique, and a run of A over (t, X_U). The proof procedes by top-down induction.

- For ε, the root of t, we have that $\rho_U(\varepsilon) = \text{choice-final}(\delta_{U_2}(t))$, since choice-final is deterministic, $\rho_U(\varepsilon)$ is unique.
- For the inner nodes x with children y and z, there exists $r \in Q$, such that $\rho_U(x) = r$ by the induction hypothesis. Then, since choice-nodes is deterministic, there exists a unique tuple (p, q, b) such that $(p, q, b) = \text{choice-nodes}(\delta_{U_2}(t^y), \delta_{U_2}(t^z), r, t(x))$. We have that $\rho_U(y) = p$ and $\rho_U(z) = q$, which verify that $\delta(p, q, (t(x), b)) = r$. We also have that $x \in X_U$ if and only if $b = 1$. This is then a valid run of A over (t, X_U)
- For a leaf x of label a, by the induction hypothesis we have that $b = \text{choice-leaves}(a, \rho_U(x))$ which is well defined since $a \in \Sigma$ and unique. We also have that $x \in X$ if and only if $b = 1$ and $\rho(x) = \text{init}((a, b))$.

The claim is proved. In particular, this run satisfies the restrictions of δ_{U_2} and it's a run over the second projection of U.

We are now ready to define a formula unif(X) that witnesses that φ is uniformisable. This formula simply states using the MSO-syntax the existence of the run ρ_U described above, *i.e.* ,

$$t \models \text{unif}(X) \text{ if and only if } X = X_U$$

Given a set of states P, from Lemma 4.8 there exists a formula u_state$_P(x)$ such that TODO: fix ustate ref

$$t \models \text{u_state}_P(x) \text{ if and only if } \delta_{U_2}(t^x) = P.$$

Let $k = |Q|$. We define 5 subformulas :

$$\begin{aligned}
\text{single_state}(X_{p_1}, \ldots, X_{p_k}) &= \forall x, \bigvee_{p_i \in \text{Nodes}(t)} X_{p_i}(x) \\
&\wedge \forall x, \bigwedge_{p_i, p_j \in \text{Nodes}(t), i \neq j} \neg(X_{p_i}(x) \wedge X_{p_j}(x))
\end{aligned}$$

$$\text{root_node}(X_{p_1},\dots,X_{p_k}) = \forall x, \text{root}(x) \to (\bigvee_{\substack{p_i \in \\ (\delta_{U_2}(\varepsilon))}} X_{p_i}(x))$$

$$\text{in_set}(X,x,b) = (\bigwedge_{b=1} X(x)) \wedge (\bigwedge_{b=0} \neg X(x))$$

$$\text{leaf_node}(X_{p_1},\dots,X_{p_k}) = \forall x \left(\text{leaf}(x) \to \bigvee_{\substack{(a,q)\in\text{init}, \\ \text{choice-leaves}(a,q)=b}} X_i(x) \wedge \text{in_set}(X,x,b) \right)$$

$$\begin{aligned} \text{inner_nodes}(X_{p_1},\dots,X_{p_k},X) \quad = \quad & \forall x \forall y, \forall z, \text{children}(x,y,z) \to \\ & \bigvee_{\substack{(P',Q',a,R')\in\delta_{U_2},\, r\in R', \\ \text{choice-nodes}(P',Q',a,r)=(p,q,b)}} \Big(\text{u_state}_{P'}(x) \wedge \text{u_state}_{Q'}(y) \\ & \wedge X_p(x) \wedge X_q(y) \wedge a(z) \wedge X_r(z) \wedge \text{in_set}(X,x,b)\Big) \end{aligned}$$

We can now define $\text{unif}(X)$:

$$\begin{aligned} \text{unif}(X) \quad = \quad & \exists X_{p_1},\dots,\exists X_{p_k}, \text{single_state}(X_{p_1},\dots,X_{p_k}) \\ \wedge \quad & \text{leaf_node}(X_{p_1},\dots,X_{p_k}) \\ \wedge \quad & \text{root_node}(X_{p_1},\dots,X_{p_k}) \\ \wedge \quad & \text{inner_nodes}(X_{p_1},\dots,X_{p_k},X) \end{aligned}$$

$\text{unif}(X)$ describes the run ρ_U over (t, X_U) and has a unique solution, X_U.

Let's show that $\text{unif}(X)$ is a uniformiser for $\varphi(X)$. If U is universal the we have that, whenever $\varphi(X)$ has a solution for a tree t, then there exists an accepting run ρ of U over t such that $\rho(\varepsilon) = (X, X')$ and such that $X \cap F \neq \emptyset$. Since the run is accepting $X' \cap F \neq \emptyset$, and we have that ρ_U is then well defined and $\text{unif}(X)$ has a solution. Since ρ_U is a run of A, then we have that X_U is also a solution of $\varphi(X)$, so $\text{unif}(X)$ is indeed a uniformiser for $\varphi(X)$.

□

Lemma 5.9. *Let $\varphi(X)$ be and MSO formula, if $\varphi(X)$ is uniformisable then the uniformiser automaton U of $\varphi(X)$ is universal.*

Proof. We will prove it by contraposition.

Let $A = (\Sigma \times \{0,1\}, Q, \text{init}, \delta, F)$ be the automaton for $\varphi(X)$ and lets suppose that $U = (\Sigma, \mathcal{P}(Q) \times \mathcal{P}(Q), \text{init}', \delta', F')$ the uniformiser automaton of φ is not universal. We will show that $\varphi(X)$ is not uniformisable.

Since U is not universal, there exists $(R, R') \in \text{Reach}_U$, such that $R \cap F \neq \emptyset$ and $R' \cap F = \emptyset$. We want to define a tree $t_{R,R'}$ such that

1. $R \subseteq \{\delta_A(t_{R,R'}, X) : X \subseteq \text{Nodes}(t_{R,R'})\}$
2. $\{\delta_A(t_{R,R'}, X) : X \text{ definable}\} \subseteq R'$

3. $[\![U]\!](t_{R,R'}) = (R, R')$

Using Lemma 5.5, if we prove that $\{\delta_A(t_{R,R'}, X) : \forall\sigma \in \mathrm{Aut}(t_{R,R'}), \sigma(X) = X\} \subseteq R'$ we also satisfy 2.

We prove by induction that $(T, T') \in \mathrm{Reach}_U$ implies all of the three conditions.

If $(T, T') = (\mathrm{init}(a, *).\mathrm{init}(a, *))$ for $a \in \Sigma$, then 1 and 3 are trivially satisfied and we have that $\mathrm{Aut}(a) = \{\mathrm{id}\}$, so 2 is also satisfied.

If $(T, T') = \delta_U((P, P'), (Q, Q'), a)$ for $a \in \Sigma$ and $P, P', Q, Q' \in \mathrm{Reach}_U$, then by the induction hypothesis there exists $t_{P,P'}$ and $t_{Q,Q'}$ that verify the hypothesis. We can construct $t_{T,T'} = a(t_{P,P'}, t_{Q,Q'})$. Lets prove that it satisfies the three properties.

Lets begin by proving 3. By definition of $[\![U]\!](\cdot)$, $[\![U]\!](t_{T,T'}) = \delta_U([\![U]\!](t_{P,P'}), [\![U]\!](t_{Q,Q'}), a)$. If we use the induction hypothesis we deduce that $\delta_U([\![U]\!](t_{P,P'}), [\![U]\!](t_{Q,Q'}), a) = \delta_U((P, P'), (Q, Q'), a)$ And by definition of (T, T'), $\delta_U((P, P'), (Q, Q'), a) = (T, T')$. 3 is then verified.

By definition, $T = \delta_A(P, Q, (a, *))$. Since by the induction hypothesis $P \subseteq \{\delta_A(t_{P,P'}, X) : X \subseteq \mathrm{Nodes}(t_{P,P'})\}$ and $Q \subseteq \{\delta_A(t_{Q,Q'}, X) : X \subseteq \mathrm{Nodes}(t_{Q,Q'})\}$ we have that $T = \delta_A(P, Q, (a, *)) \subseteq \{\delta_A(t_{T,T'}, X) : X \subseteq \mathrm{Nodes}(t_{T,T'})\}$. We have proven 1.

We will prove 2 by case distinction. Let $X \subseteq \mathrm{Nodes}(t_{P,P'})$ such that $\forall\sigma \in \mathrm{Aut}(t_{T,T'}), \sigma(X) = X$, we introduce the sets $X_1 = X \cap \mathrm{Nodes}(t_{P,P'})$ and $X_2 = X \cap \mathrm{Nodes}(t_{Q,Q'})$.

- Lets suppose $P = Q$ and $P' = Q'$, in this case $t_{P,P'} \cong t_{Q,Q'}$. TODO: explain that it's because interpret is a function Let σ be the automorphism over $\mathrm{Nodes}(T, T')$ that swaps $t_{P,P'}$ and $t_{Q,Q'}$. We have that $\sigma(X) = X$. Since $\delta(X) = X$, δ must map X_1 to X_2, and the same is true for X_2. So we have that $X_1 = X_2$ which means that $\delta(X_1) = X_1$. By construction we have that $p := \delta_A(t_{P,P'}, X_1) = \delta_A(t_{Q,Q'}, X_2)$. Using the induction hypothesis, we find that $p \in P'$ TODO: detail this. So $\delta_A(t_{T,T'}, X) = \delta_A(p, p, (a, X(\mathrm{root}))) \in T'$. TODO: define $X(\mathrm{root})$.

- Lets suppose $P' \neq Q'$ or $P \neq Q$, then $T' = \delta_{U_2}(P', Q', a) = \delta_a(P', Q', (a, *))$. We have to prove that $\{\delta_A(t_{T,T'}, X) : \forall\sigma \in \mathrm{Aut}(t_{T,T'}), \sigma(X) = X\} \subseteq T'$. Let $\sigma \in \mathrm{Aut}(t_{T,T'})$. Since $t_{P,P'} \ncong t_{Q,Q'}$, σ cannot swap $t_{P,P'}$ and $t_{P,P'}$ TODO: explain why. Since we have that $\sigma(X) = X$, then we deduce that $\sigma(X_1) = X_1$ and $\sigma(X_2) = X_2$ TODO: more details ?. Let $p := \delta_A(t_{P,P'}, X_1)$ and $q := \delta_A(t_{Q,Q'}, X_2)$. Using the induction hypothesis we deduce that $p \in P'$ and $q \in Q'$ TODO: Like in the previous case detail this?. So $\delta_A(t_{T,T'}, X) = \delta_A(p, q, (a, X(\mathrm{root}))) \in T'$.

We have proved the three properties.

Lets prove that $\varphi(X)$ is not uniformisable by exhibiting that no X verifying $t_{R,R'} \models \varphi(X)$ is definable. Let $X \in F \cap \{\delta_A(t_{R,R'}, X) : X \subset \mathrm{Nodes}(t_{R,R'})\}$, the intersection is not empty thanks to 1. TODO: more details We also know that $R' \cap F = \emptyset$ we can the deduce that $\{\delta_A(t_{R,R'}, X) : X \text{ definable}\} \cap F = \emptyset$ thanks to 2. So we know that $X \notin \{\delta_A(t_{R,R'}, X) : X \text{ definable}\} \cap F = \emptyset$ which means that no X satisfying $t_{P,P'} \models \varphi(X)$ is definable. We deduce that $\varphi(X)$ is not uniformisable. □

Theorem 5.10. *Let $\varphi(X)$ be an MSO formula, $\varphi(X)$ is uniformisable if and only if the uniformiser automaton U of $\varphi(X)$ is universal.*

Corollary 5.11. *Problem 1 is decidable and we can construct its uniformiser. TODO: reword this*

Proof. We can check if the uniformiser automaton of $\varphi(X)$ is universal using Corollary 3.15. If the test successes, then we know that φ is uniformisable and its uniformiser is $\mathrm{unif}(X)$. □

References

[Bü60] J. Richard Büchi. Weak second-order arithmetic and finite automata. *Mathematical Logic Quarterly*, 6(1-6):66–92, 1960.

[CDG+08] Hubert Comon, Max Dauchet, Rémi Gilleron, Florent Jacquemard, Denis Lugiez, Christof Löding, Sophie Tison, and Marc Tommasi. *Tree Automata Techniques and Applications*. 2008.

[CG99] Christian Choffrut and Serge Grigorieff. *Uniformization of Rational Relations*, pages 59–71. Springer Berlin Heidelberg, Berlin, Heidelberg, 1999.

[CL15] Arnaud Carayol and Christof Löding. *Uniformization in Automata Theory*. 01 2015.

[Don70] John Doner. Tree acceptors and some of their applications. *Journal of Computer and System Sciences*, 4(5):406–451, 1970.

[GS83] Yuri Gurevich and Saharon Shelah. Rabin's uniformization problem. *Journal of Symbolic Logic*, 48, 12 1983.

[Pin21] Jean-Éric Pin, editor. *Handbook of Automata Theory*, chapter 7. European Mathematical Society Publishing House, Zürich, Switzerland, 2021.

[Rab07] Alexander Moshe Rabinovich. On decidability of monadic logic of order over the naturals extended by monadic predicates. *Inf. Comput.*, 205:870–889, 2007.

[Sie68] Dirk Siefkes. J. w. thatcher and j. b. wright. generalized finite automata theory with an application to a decision problem of second-order logic. mathematical systems theory, vol. 2 (1968), pp. 57–81. *Journal of Symbolic Logic*, 37(3):619–620, 1968.

[Sie75] Dirk Siefkes. The recursive sets in certain monadic second order fragments of arithmetic. *Archiv für mathematische Logik und Grundlagenforschung*, 17(1):71–80, Mar 1975.

[Tho97] Wolfgang Thomas. *Languages, Automata, and Logic*, pages 389–455. Springer Berlin Heidelberg, Berlin, Heidelberg, 1997.